

Sortowanie plików sekwencyjnych metodą dużych buforów

Igor Sitko-Bajorski

Listopad 2025

1 Opis metody

Sortowanie plików sekwencyjnych metodą dużych buforów zakłada ograniczoną ilość pamięci operacyjnej w stosunku do rozmiaru pliku. W celu dostępu do danych w pamięci RAM wykorzystujemy zdefiniowaną ilość buforów, **oznaczoną n** , o określonej pojemności (w tym przypadku, określonym rozmiarem strony dyskowej). Ilość rekordów mieszczących się w jednym buforze nazywamy czynnikiem blokującym, oraz **oznaczamy b** , gdzie:

$$b = (\text{Rozmiar Strony Dyskowej}) / (\text{Rozmiar Rekordu})$$

Sortowanie można rozdzielić na dwa kroki:

1. Etap tworzenia serii
2. Etap złączenia serii

Etap tworzenia serii polega na:

1. wczytaniu do buforów zawartości pliku,
2. posortowaniu $n \cdot b$ rekordów w buforach, algorytmem sortowania wydajnym pamięciowo (w tym przypadku QuickSort),
3. nadpisania wczytanych danych posortowanymi. Te czynności są powtarzane do momentu osiągnięcia końca pliku.

Etap złączenia serii polega na:

1. stworzeniu tymczasowego pliku wyjściowego,
2. rozdzieleniu buforów: $n - 1$ pierwszych buforów przeznaczamy na dane wejściowe, ostatni na zapis wyników złączenia,
3. wczytaniu do buforów początków posortowanych serii,
4. utworzenie minimalnego kopca z pierwszych elementów w buforach,
5. ekstrakcję minimum, i jego zapis do bufora wyjściowego,
6. dodanie do kopca kolejnej wartości z bufora który zawierał poprzednie minimum.

W przypadku wyczerpania danych w buforze wejściowym - wczytujemy do niego kolejną stronę serii.

W przypadku osiągnięcia końca serii bufora wejściowego nie dodajemy kolejnej wartości do kopca.

W przypadku zapełnienia danych w buforze wyjściowym - zapisujemy jego zawartość do pliku wyjściowego

Kroki 5 i 6 są powtarzane do momentu opróżnienia kopca.

Te czynności są powtarzane do momentu osiągnięcia końca pliku. Pomiedzy wywołaniami zamieniamy rolami plik wejściowy i wyjściowy.

2 Specyfikacja formatu pliku

2.1 Specyfikacja rekordu

Pojedynczy rekord, opisuje obiekt na podstawie dwóch wartości masy oraz prędkości. Rekord składał się więc z dwóch wartości zmiennoprzecinkowych w formacie 64bitowym - *double*. Pojedynczy rekord opisuje obiekt na podstawie masy oraz prędkości, a jego wartość jest wyznaczana na podstawie energii kinetycznej:

$$E_k = \frac{mv^2}{2}.$$

2.2 Specyfikacja pliku

Sortowany plik składał się z zapisanych binarnie, upakowanych bez separatora, rekordów o stałej długości.

$$mvmv...mv$$

Na każdy rekord przypada wówczas $16B = 8B \text{ (masa)} + 8B \text{ (prędkość)}$. Podczas odczytu, mieszczące się na stronie pełne rekordy zostają wpisane do bufora wyjściowego jako surowe bajty, które w celu odczytania należy skonwertować na wartości *double*.

3 Prezentacja wyników działania programu

W formie podstawowej program wyświetla dwie informacje o procesie sortowania, liczbę faz etapu sortowania i etapu złączenia, oraz ilość operacji dyskowych.

W formie jawnej (verbose) wyświetla dodatkowo zawartość pliku (wartości rekordów) przed i po posortowaniu, a także zawartość pliku po każdej z faz.

W trakcie sortowania po każdym przejściu pętli w funkcjach **create_runs()** oraz **merge()** zwiększany jest licznik faz danej operacji. Klasa **FileHandler** posiada wewnętrzny licznik operacji odczytu i zapisu na dysk, a także metodę **print()**, pozwalającą na odczyt wartości rekordów w pliku.

4 Eksperyment

4.1 Sposób przeprowadzenia eksperymentu

Przyjęto wartości stałych:

$$R = 16B$$

$$b = 32$$

$$n = 10$$

Program przed rozpoczęciem sortowania generował rozmiar pliku na podstawie wzoru:

$$N = it \cdot 100,000 + (\text{rand}() \bmod 5,000),$$

gdzie it określa numer iteracji eksperymentu, $0 \leq it \leq 10$.

Po określeniu rozmiaru, generował plik wejściowy o zadanej wielkości i zapisywał rozmiar (N) do pliku opisującego eksperyment. Następnie zapisywał czas startu wywołania funkcji **sort()** i uruchamiał proces sortowania. W tym kroku, program dopisywał do pliku ilość wykonanych w tym procesie operacji dyskowych, będących sumą operacji r/w na sortowanym pliku i na pliku tymczasowym etapu złączenia. Dopisywał również ilość faz poszczególnych etapów. Po tym zapisywał czas końca wywołania, i dopisywał czas trwania wywołania do pliku opisującego eksperyment.

4.2 Wyniki eksperymentu

4.2.1 Ilość operacji r/w

Etap sortowania zajmie $\frac{N}{b}$ operacji odczytu danych z pliku i tyle samo operacji zapisu - musimy 1 raz wczytać i zapisać cały plik.

Każda iteracja etapu złączenia również zajmie $\frac{N}{b}$ operacji odczytu i zapisu danych z/do pliku.

Po pierwszej iteracji rozmiar serii będzie wynosił $n \cdot b$. Po drugiej będzie to już $n^2 \cdot b$ - łączymy ($n - 1$) serii, ale $n \gg 1$, więc można założyć że $n \approx n - 1$. Z tego wynika że tych etapów będzie ok.

$$\lceil \log_n(\frac{N}{nb}) \rceil = \lceil \log_n(\frac{N}{b}) \rceil - 1$$

Liczbę operacji dyskowych w zależności od rozmiaru pliku można oszacować więc wzorem:

$$\frac{2N}{b} \cdot \log_n(\frac{2N}{b}) = \frac{2N}{b \cdot \log_2(n)} \cdot \log_2(\frac{N}{b})$$

Dokładny wzór:

$$\lceil \frac{2N}{b} \rceil \cdot (1 + \lceil \log_{n-1}(\frac{N}{nb}) \rceil)$$

Różnice między wartością aproksymowaną 1 wzorem, a rzeczywistą pokazuje poniższy wykres.

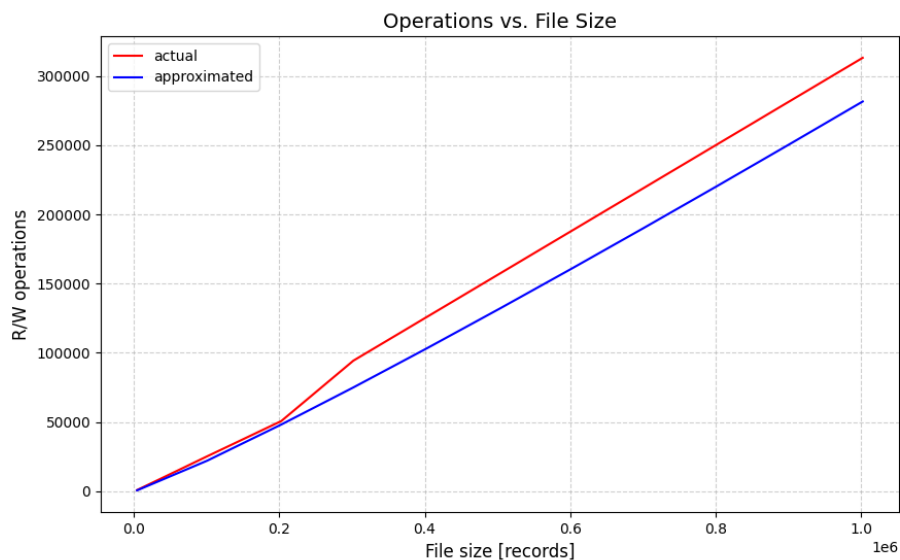


Figure 1: Rzeczywiste i aproksymowane ilości operacji dyskowych. Na osi X rozmiar pliku N , na osi Y liczba operacji dyskowych

Na wykresie widać że zgodnie z przewidywaniem wykres rośnie prawie liniowo. Zaobserwowane ilości operacji dyskowych dla mniejszych rozmiarów pokrywają się, natomiast zwiększając rozmiar pliku, od ok. 100.000 rekordów występują rozbieżności.

Różnice między wartością aproksymowaną 2 wzorem, a rzeczywistą pokazuje poniższy wykres.

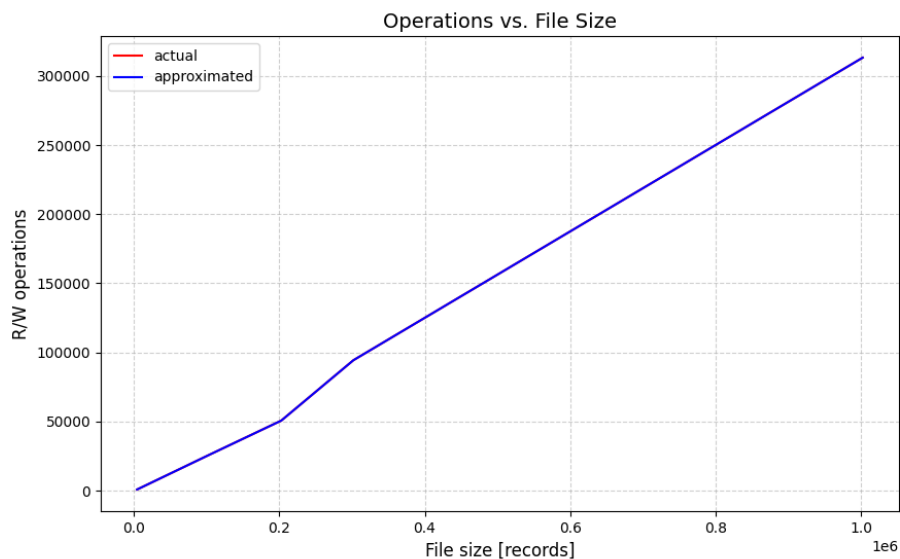


Figure 2: Rzeczywiste i aproksymowane ilości operacji dyskowych. Na osi X rozmiar pliku N , na osi Y liczba operacji dyskowych

Po dodaniu zaokrągleń i dokładnego logarytmu o podst. $n - 1$ do drugiej części, wzór na aproksymację dokładnie pokrywa się z obserwacjami.

4.2.2 Ilość faz

Ilości faz sortowania, to kolejna metryka którą przedstawia poniższy wykres.

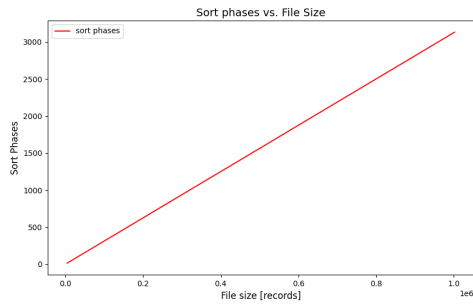


Figure 3: Fazy sortowania w zależności od rozmiaru pliku

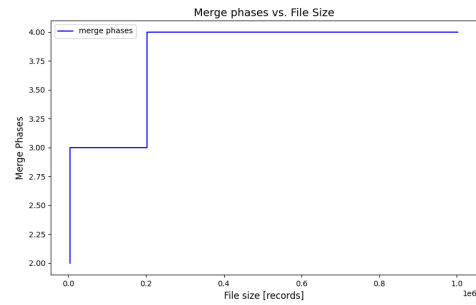


Figure 4: Fazy złączenia w zależności od rozmiaru pliku

Ilość faz sortowania zwiększa się liniowo wraz z pojemnością pliku co zgadza się z teorią wzoru.

Ilość faz złączenia zgadza się z wartością teoretyczną.

4.2.3 Czas wykonania

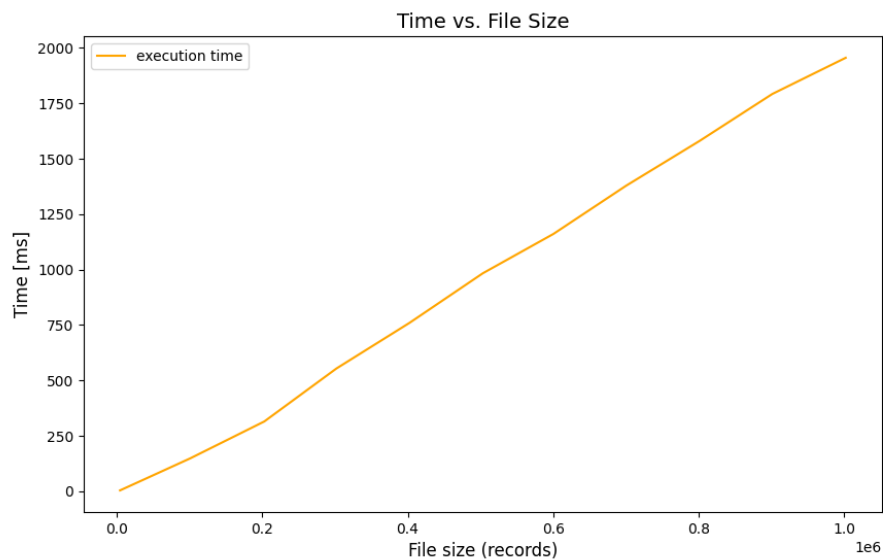


Figure 5: Rzeczywiste i aproksymowane ilości operacji dyskowych. Na osi X rozmiar pliku na osi Y czas wykonania

Czas wykonania rośnie liniowo w zależności od rozmiaru pliku.

4.3 Wnioski

4.3.1 Ilość operacji r/w

Wzór bez zaokrągleń generuje większe rozbieżności dla operacji r/w, ponieważ założenie $n \gg 1$ nie jest spełnione. Dla $n = 10$, 1 to już 10% odchylenie. Ponadto błędy nawarstwiają się w przypadku zmiany podstawy logarytmu na dwójkowy. Wykres posiada również załamanie spowodowane przeskokiem z ok. 5,000 próbek na ok. 100,000.

Stosując za to dokładny wzór jesteśmy w stanie, za każdym razem dokładnie określić ilość operacji dyskowych zgodną z rzeczywistością, w każdym punkcie wynik jest zgodny.

4.3.2 Ilość faz

Ilość faz sortowania wzrasta liniowo z podaną zależnością. Ilość faz jest równa $\frac{N}{nb}$, bo w każdym kroku odczytamy nb danych (pojemność jednego bufora razy ilość buforów).

Ilość faz złączenia wzrasta logarytmicznie, zgodnie z podaną zależnością $\lceil \log_{n-1}(\frac{N}{nb}) \rceil$, a obserwacje zgadzają się ze wskazaniem teoretycznym.

4.3.3 Czas wykonania

Czas wykonania algorytmu zależy głównie od ilości operacji dyskowych, ponieważ to one są obciążone największym opóźnieniem. Pokrywa się to z danymi na wykresie, który wzrasta liniowo podobnie do ilości operacji odczytu i zapisu.